

Autonomous supply chain recovery

Problem 6 | Smart Automation | Agentic AI Hackathon, Tech Zephyr 4.0, IIT Bhubaneswar

Problem and target users

Retail operations teams must recover a delayed shipment while inventory, supplier availability and service constraints keep changing. A suggested route is insufficient: recovery requires valid reservations, feedback from execution and a verified end state.

Proposed solution

RouteGuard is a runnable symbolic planning agent inside a transactional logistics sandbox. It observes the shortage, retrieves supplier and route evidence, searches feasible allocations, commits reservations and verifies the resulting inventory and financial state. Failures trigger fresh observation and replanning; impossible goals are escalated.

Complete demonstration

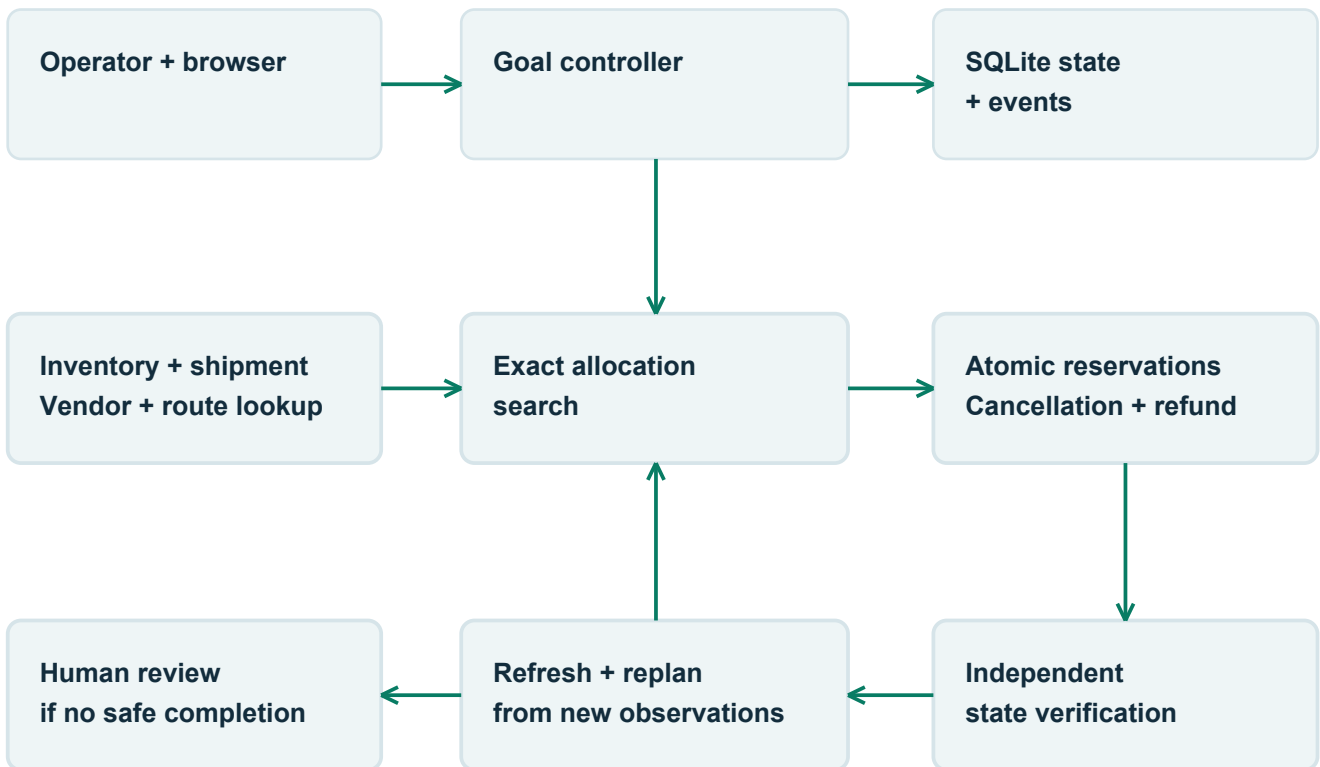
The store needs 100 recovery units beyond its 20 on-hand units. The original shipment arrives in 36 hours, missing the 24-hour target. The agent must remain within INR 14,000 and 110 kg CO2e. A supplier disappears after planning, then another route closes after reservations are committed.

Final result	Achieved	Constraint
Recovery units	100	100 required
Net committed cost	INR 11,700	INR 14,000 maximum
Modeled carbon	38 kg CO2e	110 kg maximum
Latest arrival estimate	22 hours	24-hour deadline

Result: two replans and ten passing verification checks. All numbers describe a synthetic environment; verified reservations are not proof of physical delivery.

System architecture

A closed feedback loop connects the operator, planning tools, sandbox actions and persistent memory.



Decision and execution model

Choose nonnegative integer quantities in ten-unit lots. Enforce demand, stock, availability, delivery deadline, remaining budget and remaining carbon as hard constraints. Among feasible allocations, minimize cost, then carbon, then latest arrival. Existing valid reservations reduce the remaining shortage and resource allowances.

The commit tool rechecks the catalog and revision before any mutation. One SQLite transaction persists inventory deductions, orders, ledger entries, receipts and the event. Idempotency keys prevent a repeated reservation request from charging twice. Stale data triggers a new read and a new plan.

The controller is model-based symbolic AI with an explicit finite policy. It uses no LLM, RAG, cloud account or external API. The demonstration faults are scenario fixtures; allocations and verification are computed at runtime. Judges can inject alternative outages through the UI.

Evaluation and robustness

Reproducible tests distinguish feasible recovery from correctly identified impossibility.

Evidence

Check	Measured result
Independent oracle agreement	80 / 80 generated cases
Feasible cases independently verified	31 / 31
Infeasible cases identified	49 / 49
Cheapest-first baseline feasible cases	28 / 80
Automated correctness/integration tests	17 passing
Runtime state checks	10 checks per final audit

Benchmark seed: 1742838. The separate dynamic-programming oracle verifies exact feasibility and the lexicographic objective without calling the agent optimizer. Every generated input and result is included in the evaluation JSON. The baseline is a simple cheapest-first policy without global carbon search; this is a limited sandbox comparison.

Failure and integrity coverage

Tests cover stale revisions, atomic batches, unavailable suppliers, duplicate requests, forged prices, changed demand, saved-state restart, concurrent steps, live outages, invalid HTTP input, ledger tampering and stock/price inconsistencies. An integrity failure blocks a success claim and requests operator review.

Reproduce locally

```
python server.py
```

```
python -m unittest discover -s tests -v
```

```
python evaluate.py
```

```
python run_demo.py --scenario double_disruption
```

Open <http://127.0.0.1:8765>. Requires Python 3.10+ and a browser. No package installation or credentials are required.

Why this problem and what comes next

The project targets the published criteria with behavior that can be reproduced and inspected.

Selection rationale

Problem 6 explicitly permits simulated logistics. It lets us own the data, implement meaningful state-changing tools and prove objective outcomes without private records. Customer resolution and IAM mitigation were strong alternatives; supply recovery gives the clearest visible trade-off among cost, carbon and delivery while demonstrating repeated adaptation.

Published criterion	Weight	RouteGuard evidence
Autonomy	25%	Goal and feedback-driven decisions
Tool interaction	15%	Reservations, refunds, state changes
Adaptation	15%	Two disruptions and residual replans
Implementation	15%	Exact search and SQLite transactions
Relevance / innovation	10%	Preserve valid existing commitments
Functionality / UX	10%	Working console and evidence export
Verification / robustness	10%	Oracle benchmark and failure tests

Limits and expected impact

One SKU, one store, five offers and ten-unit lots. ETAs are snapshot estimates. Pre-dispatch cancellations are fully refundable with no sunk carbon. No real ERP integration, audited emissions, physical delivery guarantee or measured business saving is claimed. A production version needs scalable optimization, authentication, real adapters and uncertain transit/cancellation costs.

Expected impact: operators can inspect why an action was chosen and whether recovery actually satisfies the modeled objective. Qualification cannot be predicted from the rubric alone.

Official sources (reviewed 12 September 2026)

Round 1 problem statements and evaluation rubric

Unstop competition listing and submission requirements